

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What is the output?

```
def fun():
    try:
        return 10
    finally:
        return 20
```

```
print(fun())
```

A)RuntimeError

B)10

C)20

D)None

Q2. What does the following function return?

```
def fun():
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

A)Hello

B)None

C>Error

D>Hello None

Q3. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

A)There is no difference

B)First imports only sqrt

C)First provides the module namespace; second imports sqrt directly

D)Both import only sqrt

Q4. What will be printed?

```
x = 10
```

```
def outer():
```

```
    x = 20
```

```
    def inner():
```

```
        print(x)
```

```
    x = 30
```

```
    inner()
```

```
outer()
```

A)10

B)NameError

C)20

D)30

Q5. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

A)6

B)6.0

C)36

D)NameError

Q6. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a
```

```
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)ImportError



B)Module loaded

Module loaded

10

C)10

10

D)Module loaded

10

Q7. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
numbers)
    )
)
```

```
print(result)
```

A)[1, 3, 5]

B)[4, 8]

C)[2, 4, 6, 8, 10]

D)[2, 4]

Q8. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

A)SyntaxError

B)NameError

C)10

D)20

Q9. What happens here?

```
try:
```

```
    raise ValueError("Wrong value")
```

```
except ValueError as e:
```

```
    print(e)
```

A)e

B)Nothing

C)ValueError

D)Wrong Value

Q10. What is the output?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        return x + 5
```

```
    return inner
```

```
f = outer()
```

```
print(f())
```

A)15

B)NameError

C)None

D)10

Q11. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

```
print(fun())
```

A)Finally followed by 10

B)Finally only

C)10 followed by Finally

D)10 only

Q12. Why is

```
try:
```

```
    print(10 / 0)
```

```
except Exception:
```

```
    print("A")
```

```
except ZeroDivisionError:
```

```
    print("B")
```

codes's second handler never reached?

A)ZeroDivisionError is not an exception

B)except blocks execute randomly

C)Exception is a superclass of ZeroDivisionError

D)ZeroDivisionError occurs before Exception

Q13. What happens?



```
def fun(a, b):  
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)20
- C)TypeError
- D)30

Q14. Which object keeps track of modules that have already been imported during the current Python process?

- A)sys.imports
- B)sys.modules
- C)sys.path
- D).__module__

Q15. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

- A)1
- 1
- 1
- 1
- B)0
- 1
- 0
- 2
- C)1
- 2
- 3
- 4

D)1

2

1

3

Q16. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)The local file can shadow the intended standard-library module
- B)Python automatically renames the local file
- C)random cannot be imported
- D)Python will always reject it

Q17. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)20
- B)NameError
- C)None
- D)10

Q18. What is the output?

```
try:
```

```
    x = int("abc")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)A C

C)B C

D)B

Q19. What will be printed?

```
def fun():
```

```
    print("A")
```

```
    return
```



print("B")

fun()

- A)A
- B)Nothing
- C)B
- D)A B

Q20. What will be the output?

x = 10

def fun():

print(x)

x = 20

fun()

- A)20
- B)NameError
- C)10
- D)UnboundLocalError

Q21. What is the output?

def fun():

try:

print("A")

raise ValueError

except ValueError:

print("B")

return 10

else:

print("C")

return 20

finally:

print("D")

print(fun())

- A)A
- C
- D
- 20
- B)A
- B
- 10
- D
- C)A
- B
- D

10

D)A

B

C

D

10

Q22. What will be printed?

def fun(a, b):

return a + b

print(fun(10, b=20))

- A)30
- B)SyntaxError
- C)TypeError
- D)20

Q23. What is the purpose of this statement?

if __name__ == "__main__":

- A)It identifies code intended to run when the file is executed directly
- B)It prevents syntax errors
- C)It creates the main function
- D)It prevents all imports

Q24. What will be the output?

def fun(x):

try:

if x < 0:

raise ValueError("negative")

return x

except ValueError:

return 100

finally:

print("cleanup")

print(fun(-5))

- A)cleanup
- 100
- B)cleanup
- 5
- C)negative
- cleanup
- D)100
- cleanup

Q25. What will happen?



```
def fun(a, b=10):  
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)15
- B)SyntaxError
- C)TypeError
- D)25

Q26. What happens here?

```
def fun(a):  
    return a
```

```
print(fun())
```

- A)NameError
- B)0
- C)TypeError
- D)None

Q27. What is the output?

```
f = lambda x: x + 5
```

```
print(f(10))
```

- A)15
- B)10
- C>Error
- D)5

Q28. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

- A)Nothing
- B)B
- C)A and B
- D)A

Q29. What is the output?

```
def fun(x):  
    x += 10  
    return x
```

```
x = 5  
y = fun(x)
```

```
print(x, y)
```

- A)5 5
- B)5 15
- C)15 5
- D)15 15

Q30. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

```
print(fun(30))
```

- A)30
- B)10
- C)20
- D)-10

Q31. What will normally happen?

```
import math  
import math
```

- A)The module is normally loaded once and reused
- B)SyntaxError
- C)ImportError
- D)The module is normally loaded and executed twice

Q32. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

```
print(result)
```

- A)9
- B)24
- C)10
- D)0

Q33. What will be the output?

```
import math
```



```
print(math.sqrt(25))
```

A)sqrt(25)

B)5.0

C)5

D)25

Q34. What will be printed?

```
try:
```

```
    x = int("10")
```

```
except ValueError:
```

```
    print("A")
```

```
else:
```

```
    print("B")
```

```
finally:
```

```
    print("C")
```

A)A

B)B

C)A C

D)B C

Q35. What will be printed?

```
def fun(x=[]):
```

```
    x.append(10)
```

```
    return x
```

```
a = fun()
```

```
b = fun()
```

```
print(a)
```

```
print(b)
```

A)Error

B)[10]

[10]

C)[10]

[10, 10]

D)[10, 10]

[10]

Q36. What happens when this function is called?

```
def fun(a=10, b):
```

```
    return a + b
```

A)TypeError

B)NameError

C)It executes correctly

D)SyntaxError

Q37. What will be printed?

```
class InvalidAgeError(Exception):
```

```
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is too low")
```

```
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

A)Exception

Age is too low

B)InvalidAgeError

15

C)Nothing, because custom exceptions cannot be caught

D)InvalidAgeError

Age is too low

Q38. What will be printed?

```
try:
```

```
    print(10 / 0)
```

```
except ZeroDivisionError:
```

```
    print("A")
```

```
except Exception:
```

```
    print("B")
```

A)A

B)Nothing

C)B

D)A B

Q39. What is the output?

```
def fun(x):
```

```
    if x == 0:
```

```
        return
```

```
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)None



C)3

D)5

Q40. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)Handled

B)NameError

C)RuntimeError cannot propagate through functions

D)TypeError

Q41. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

A)TypeError

B)RecursionError

C)3

D)6

Q42. What is the output?

```
try:  
    print(10 / 0)  
except Exception:  
    print("A")  
except ZeroDivisionError:  
    print("B")
```

A)B

B)A

C)SyntaxError

D)A B

Q43. Suppose sample.py contains:

```
x = 10
```

```
def display():  
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)  
sample.display()
```

A)Only Hello

B)x

display

C)10

Hello

D>Error

Q44. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

```
variable = 10
```

```
fun()
```

A)UnboundLocalError

B)10

C)NameError

D)50

Q45. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```



A)20

10

B)30

30

C)20

30

D)30

20

Q46. What is the output?

```
def fun(x):  
    return x * 2
```

```
a = fun  
print(a(5))
```

A)10

B)func(5)

C>Error

D)5

**Q47. if test.py has
print("A")**

```
if __name__ == "__main__":  
    print("B")
```

**If test.py is executed directly, what
will be printed?**

A)A

B)A followed by B

C)B

D)Nothing

Q48. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

```
print(outer())
```

A)10

B)5

C)15

D)UnboundLocalError

Q49. What happens here?

```
class MyError(Exception):  
    pass
```

```
def test():  
    raise MyError("Something went  
wrong")
```

```
try:  
    test()  
except MyError as e:  
    print(type(e).__name__)
```

A)TypeError

B)MyError

C)Exception

D)Something went wrong

Q50. What is the output?

```
try:  
    raise ValueError  
except ValueError:  
    print("A")  
    raise
```

A)A, and execution ends normally

B)No Output

C)A, followed by propagation of the same
exception

D)B

